Programming with C++

COMP2011: C++ Basics II

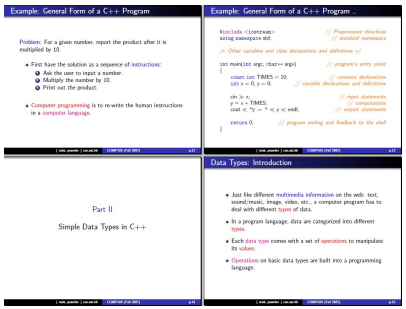
Gary Chan
James Cheung
Cindy Li

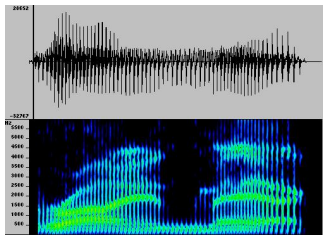
Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China



Part I

More Basic Data Types in C++









C++ Basic Types

Types	Common Size(#bytes on a 32-bit machine)	Value Range
bool	1	{ true, false }
char	1	[-128, 127]
short	2	[-32768, 32767]
int	4	$[-2^{31}, 2^{31}-1]$
long	4	$[-2^{31}, 2^{31}-1]$
float	4	$\pm$ [1.17549E-38, 3.40282E+38]
double	8	$\pm$ [2.22507E-308, 1.79769E+308]

- Not all numbers of a type can be represented by a computer.
- It depends on how many bytes you use to represent it: with more bytes, more numbers can be represented.

Integers

- Type names: short (int), int, long (int), long long (int)
- Their sizes depend on the CPU and the compiler.
- ANSI C++ requires:
  size of short $\leq$ size of int $\leq$ size of long $\leq$ size of long long
- e.g., What are the numbers that can be represented by a 2-byte short int?
- Each integral data type has 2 versions:
  - signed version: represents both +ve and -ve integers.
    e.g. signed short, signed int, signed long
  - unsigned version: represents only +ve integers.
    e.g. unsigned short, unsigned int, unsigned long
- signed versions are the default.
- Obviously unsigned int can represent 2 times more +ve integers than signed int.

Floating-Point Data Types

- Floating-point numbers are used to represent real numbers and very large integers (which cannot be held in long long).
- Type names:
  - float for single-precision numbers.
  - double for double-precision numbers.
- Precision: For decimal numbers, if you are given more decimal places, you may represent a number to higher precision.
  - for 1 decimal place: 1.1, 1.2, 1.3, ... etc.; can't get 1.03.
  - for 2 decimal places: 1.01, 1.02, 1.03, ... etc.; can't get 1.024.
- In scientific notation, a number has 2 components. e.g., 5.16E-02
  - mantissa: 5.16
  - exponent: -2
- More mantissa bits $\Rightarrow$ higher precision.
- More exponent bits $\Rightarrow$ larger real number.

Integer Arithmetic and Floating-Point Arithmetic

- Arithmetic expressions involving *only* integers use integer arithmetic.
- Arithmetic expressions involving *only* floating-point numbers use floating-point arithmetic.
- For $+$, -, $\times$ operations, results should be what you expect.
- However, integer division and floating-point division may give different results. e.g.,
  - $10/2 = 5$ and $10.0/2.0 = 5.0$
  - $9/2 = 4$ and $9.0/2.0 = 4.5$
  - $4/8 = 0$ and $4.0/8.0 = 0.5$

Boolean Data Type

- Type name: bool.
- Used to represent the truth value, true or false of logical (boolean) expressions like:

$$a > b \qquad x + y == 0 \qquad \text{true \&\& false}$$

- Since C++ evolves from C, C++ follows C's convention:
  - zero may be interpreted as false.
  - non-zero values may be interpreted as true.
- However, since internally everything is represented by 0's and 1's,
  - false is represented as 0.
  - true is represented as 1.
- Even if you put other values to a bool variable, its internal value always is changed back to either 1 or 0.

Example: Output Boolean Values

```
#include <iostream> /* File: boolalpha.cpp */
using namespace std;

int main()
{
    bool x = true;
    bool y = false;

    // Default output format of booleans
    cout << x << " && " << y << " = " << (x && y) << endl << endl;

    cout << boolalpha;   // To print booleans in English
    cout << x << " && " << y << " = " << (x && y) << endl << endl;

    cout << noboolalpha; // To print booleans in 1 or 0
    cout << x << " && " << y << " = " << (x && y) << endl;

    return 0;
}
```

Example: Use of bool Variables

```
#include <iostream> /* File: bool-blood-donation.cpp */
using namespace std;

int main()
{
    char donor_blood_type, recipient_blood_type;
    bool exact_match, match_all;

    cout << "Enter donor's bloodtype: A, B, C (for AB), and O. ";
    cin >> donor_blood_type;
    cout << "Enter recipient's bloodtype: A, B, C (for AB), and O. ";
    cin >> recipient_blood_type;

    exact_match = (donor_blood_type == recipient_blood_type);
    match_all = (donor_blood_type == 'O');

    if (exact_match || match_all)
        cout << "Great! A donor is found!" << endl;
    else
        cout << "Keep searching for the right donor." << endl;

    return 0;
}
```

Part II

Type Checking and Type Conversion



Type Checking and Coercion

Analogy:

Blood Types	
Receiver	Donor
A	A, O
B	B, O
AB	A, B, AB, O
O	O

- For most languages, data types have to be matched during an operation ⇒ type checking.
- However, sometimes, a type is made compatible with a different type ⇒ coercion.

Operand Coercion

Coercion is the automatic conversion of the data type of operands during an operation.

- Example: $3 + 2.5 \Rightarrow$ int + double.
- The C++ compiler will automatically change it to
$$3.0 + 2.5 \Rightarrow \text{double} + \text{double}$$
- Thus, the integer 3 is coerced to the double 3.0.

Example: Convert a Small Character to Capital Letter

```
char small_y, big_y;
cin >> small_y;                  // Character in small case
big_y = small_y + 'A' - 'a'; // Character in big case
```

Here big_y, small_y, 'A', and 'a' are "coerced" by "promoting" it to int before addition. The result is converted back (or coerced) to char.

Automatic Type Conversion During Assignment

Examples

```
float x = 3.2;          // Initialize x with 3.2 by assignment
double y = 5.7;         // Initialize y with 5.7 by assignment

short k = x;            // k = ?
int n;
n = y;                  // n = ?
```

- Since float|double can hold numbers bigger than short | int, the assignment of k and n in the above program will cause the compiler to issue a warning — not an error.

Compiler Warnings

a.cpp:9: warning: converting to 'short int' from 'float'
a.cpp:11: warning: converting to 'int' from 'double'

Automatic Type Conversion During Assignment ..

- A narrowing conversion changes a value to a data type that might not be able to hold some of the possible values.
- A widening conversion changes a value to a data type that can accommodate any possible value of the original data.
- C++ uses truncation rather than rounding in converting a float|double to short | int | long.

Manual Type Conversion (Casting)

```
int k = 5;
int n = 2;
float x = n/k;          // What is the value of x?
```

- In the above example, one can get x = 0.4 by manually converting *n* and/or *k* from int to float|double.

Syntax: static_cast for manual type casting

static_cast<data-type> (value)

- No more warning messages on narrowing conversion.

```
int k = 5, n = 2;
float x = static_cast<double>(n)/k;
float y = n/static_cast<double>(k);
float z = static_cast<double>(n)/static_cast<double>(k);
```

Part III

Constants



Literal Constants

- Constants represent fixed values, or permanent values that cannot be modified (in a program).
- Examples of literal constants:
  - char constants: 'a', '5', '\n'
  - string constants: "hello world", "don't worry, be happy"
  - int constants: 123, 456, -89
  - double constants: 123.456, -2.90E+11

Symbolic Constants

- A symbolic constant is a named constant with an identifier name.
- The rule for identifier names for constants is the same as that for variables. However, by convention, constant identifiers are written in capital letters.
- A symbolic constant must be defined and/or declared before it can be used. (Just like variables or functions.)
- Once defined, symbolic constants cannot be changed!

Syntax: Constant Definition

const <data-type> <identifier> = <value> ;

Example

```
const char BACKSPACE = '\b';
const float US2HK = 7.80;
const float HK2RMB = 0.86;
const float US2RMB = US2HK * HK2RMB;
```

Why Symbolic Constants?

Compared with literal constants, symbolic constants are preferred because they are

- more readable. A literal constant does not carry a meaning. e.g. the number 320 cannot tell you that it is the enrollment quota of COMP2011 in 2015.

```
const int COMP2011_QUOTA = 320;
```

- more maintainable. In case we want to increase the quota to 400, we only need to make the change in one place: the initial value in the definition of the constant COMP2011_QUOTA.

```
const int COMP2011_QUOTA = 400;
```

- type-checked during compilation.

Remark: Unlike variable definitions, memory is not allocated for constant definitions with only few exceptions.

Example: Use of Symbolic Constants

```
#include <iostream> /* File: symbolic-constant.cpp */
include <cmath> // For calling the ceil() function
using namespace std;

int main()
{
    const int COMP2011_QUOTA = 320;
    const float STUDENT_2_PROF_RATIO = 100.0;
    const float STUDENT_2_TA_RATIO = 40.0;
    const float STUDENT_2_ROOM_RATIO = 100.0;

    cout << "COMP2011 requires "
         << ceil(COMP2011_QUOTA/STUDENT_2_PROF_RATIO)
         << " instructors, "
         << ceil(COMP2011_QUOTA/STUDENT_2_TA_RATIO)
         << " TAs, and "
         << ceil(COMP2011_QUOTA/STUDENT_2_ROOM_RATIO)
         << " classrooms" << endl;

    return 0;
}
```

Part IV

Array

What is an Array?



- Array is a collection of homogeneous objects: objects of the same type. e.g., a collection of int, char, double, ..., or user-defined types.
- Exception: The array elements cannot be reference variables.

Motivation from Programming Point of View

- A program to sort 3 integers: a, b, c
- A program to sort 6 integers: a, b, c, d, e, f
- How about a program to sort 10,000 integers? Are you going to create variable names for the 10,000 different integers?
- Array is designed to solve this problem: you only need one identifier name to address all the 10,000 integers, and there is a way to refer to each of them.
- It can solve problems like: read a list of student names, and sort them in alphabetical order.
- In an Excel file, each column/row is basically an array so that you can do some common operations (like average, max, min, count) on it.

C++ 1-Dimensional Array

Syntax: Definition of a 1D Array

<data-type> <array-name> [<size>] ;

- <size> should be a positive constant. It can be a constant expression too.

Examples

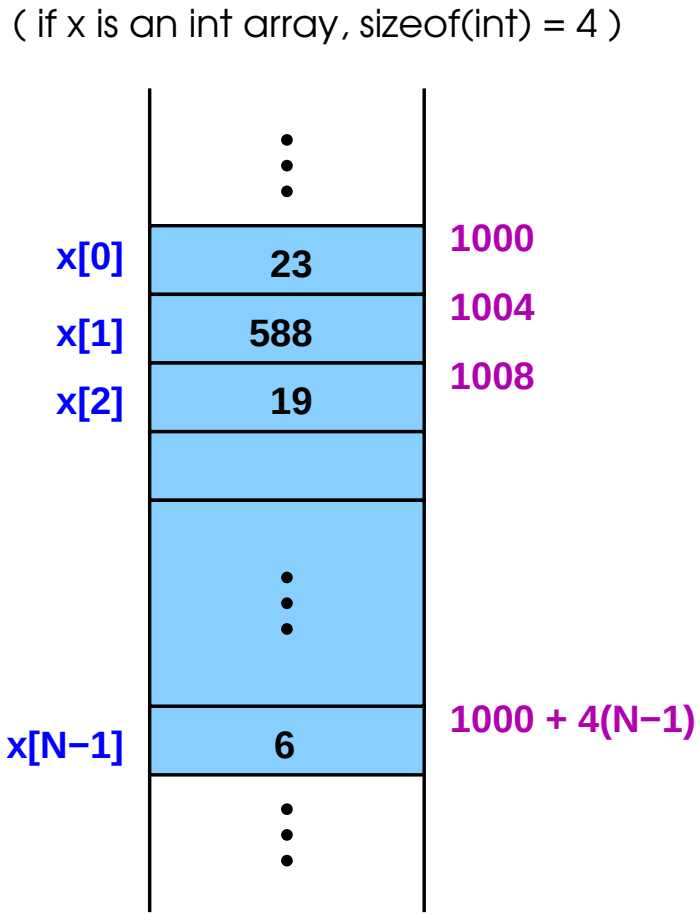
```
int number[10000]; // An array of 10,000 uninitialized integers

const int NUM_STUDENTS = 335;
char gender[NUM_STUDENTS];     // An array of 335 char
float score[NUM_STUDENTS + 1]; // An extra element to hold the mean

int n = 3;
double x[n];       // Compilation error on VC++: size is NOT a constant

int value[-4];     // Compilation error: array size cannot be -ve
```

Subscripting: Access to Each Array Element



- A 1D array is an ordered list of elements.
- Successive elements are stored in contiguous memory.
- To access an element, use the subscript operator [] with an array index.
- For an array of size N, the indices run from $0, 1, 2, \ldots, N-1$.
- Each array element is treated like a regular variable:
  - you may assign a value to it
  - you may assign its value to another variable
  - you may pass it by value or reference to a function

Array and Control

- Array works particularly well with loops: e.g., use a for-loop to access and manipulate each array element in turn.
- This is not a coincidence, but part of the C++ language design.

Examples

```
int y;                      // A regular int variable
int x[3];                   // An array of 3 int numbers

x[0] = 34;                  // Array indices start from zero in C++
x[1] = 289;
x[2] = 75;                  // Index of the last element is 2 NOT 3!

y = x[2];                   // Now both y and x[2] are 75
max(x[2], x[0]);            // Pass array elements by value
swap(x[1], x[0]);           // Pass array elements by reference

for (int j = 0; j < 3; j++)
    x[j] *= 3;              // Triple each element of an array
```

Example: Manipulate an Array of Scores using for Loop

```
#include <iostream> /* array-mean.cpp */
using namespace std;

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];
    float sum_score = 0.0;      // Don't forget initializing the sum

    for (int j = 0; j < NUM_STUDENTS; ++j)
    {
        cin >> score[j];
        sum_score += score[j]; // Accumulate the scores
    }

    cout << "mean score = " << sum_score/NUM_STUDENTS << endl;
    return 0;
}
```

Example: Manipulate an Array of Scores using for Loop ..

```
#include <iostream> /* array-max.cpp */
using namespace std;

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];
    // Read in the first student's score. Assume students >= 1
    cin >> score[0];
    float max_score = score[0]; // A good way to initialize max score

    for (int j = 1; j < NUM_STUDENTS; ++j)
    {
        cin >> score[j];
        if (max_score < score[j])
            max_score = score[j];
    }

    cout << "max score = "  << max_score << endl;
    return 0;
}
```

Wrong Subscript: Common Reason for Segmentation Fault

- C++ compiler does not automatically check that an array index is out of bound.
- That is, for an array of size N, the compiler won't check if it is subscripted with an index between 0 and $N-1$, neither at compile-time nor run-time.
- There is no compilation error for the following codes:

```
int x[10]; x[-2] = 5; x[100] = 9;
```

- When the codes are run, `x[-2] = 5;` will put the value 5 to the memory space which is 2×4 bytes (size of 2 int) before the array x. Similarly, `x[100] = 9;` will put the value 9 to the memory space which is 90×4 bytes beyond the array.
- This is a common cause of the run-time error called segmentation fault — your program trespasses into memory locations that may not belong to it.

Array Initialization

- Just like any local variable, when an array is defined, its elements are not initialized automatically.

Syntax: Define and Initialize a 1D Array Simultaneously

<data-type> <array-name> [<size>]
= { <value$_0$>, <value$_1$>, . . . , <value$_{<size>-1}$> } ;

- If there are fewer values than the array size, the unspecified values will be zeros.
- It is a compilation error if there are more values than the array size.
- If you leave out the array size in the array initialization, the compiler will count the number of initializing values and uses that as the array size.
- Once defined, you cannot assign values to an array using the initialization syntax.

Example: Array Initialization

```
int a[5] = {1, 2, 3, 4, 5};
/* Same as

 int a[5];
 a[0] = 1; a[1] = 2; a[2] = 3; a[3] = 4; a[4] = 5;
*/

int b[5] = {1, 2};     // => 1, 2, 0, 0, 0
int c[5] = {};         // => 0, 0, 0, 0, 0
int d[] = {1, 2, 3};   // Compiler determines the size=3 automatically
int e[3];

// Compilation error:
// can't assign values to an array using the { } syntax
e = {5, 6, 7};

// Compilation error: can't declare an array of references
double x = 1.5, y = 2.5, z = 3.5;
double& s[] = {x, y, z};
```

Common Mis-uses of an Array

While each array element can be treated as a simple variable, the whole array, as represented by the array identifier, cannot.

Examples: Correct and Incorrect Uses of Arrays

```
int x[] = {1, 2, 3, 4, 5 };
int y[] = {6, 7, 8, 9, 0 };
int z[5];

/* Incorrect way */
// Cannot assign to array elements using the initialization syntax
x = {5, 4, 3, 2, 1};

x = 8;        // x is not an integer! Its elements are.
x += 2;       // x is not an integer! Its elements are.
x = y;        // No assignment between 2 arrays
z = x + y;    // Cannot +, -, *, / on the array, but only its elements

/* Correct way; what does each for-statement do? */
for (int j = 0; j < 5; ++j) x[j] = 5 - j;
for (int j = 0; j < 5; ++j) x[j] = 8;
for (int j = 0; j < 5; ++j) x[j] += 2;
for (int j = 0; j < 5; ++j) x[j] = y[j];
for (int j = 0; j < 5; ++j) z[j] = x[j] + y[j];
```

Part V

Further Readings and Examples

Find Out the Size of a Data Type using sizeof

```
#include <iostream> /* File: value.cpp */
using namespace std;

int main()
{
    cout << "sizeof(bool) = " << sizeof(bool) << endl;
    cout << "sizeof(char) = " << sizeof(char) << endl;
    cout << "sizeof(short) = " << sizeof(short) << endl;
    cout << "sizeof(int) = " << sizeof(int) << endl;
    cout << "sizeof(long) = " << sizeof(long) << endl;
    cout << "sizeof(long long) = " << sizeof(long long) << endl;
    cout << "sizeof(float) = " << sizeof(float) << endl;
    cout << "sizeof(double) = " << sizeof(double) << endl;
    cout << "sizeof(long double) = " << sizeof(long double) << endl;

    return 0;
}
```

Size of Basic Types on 2 Computers

on a 32-bit machine

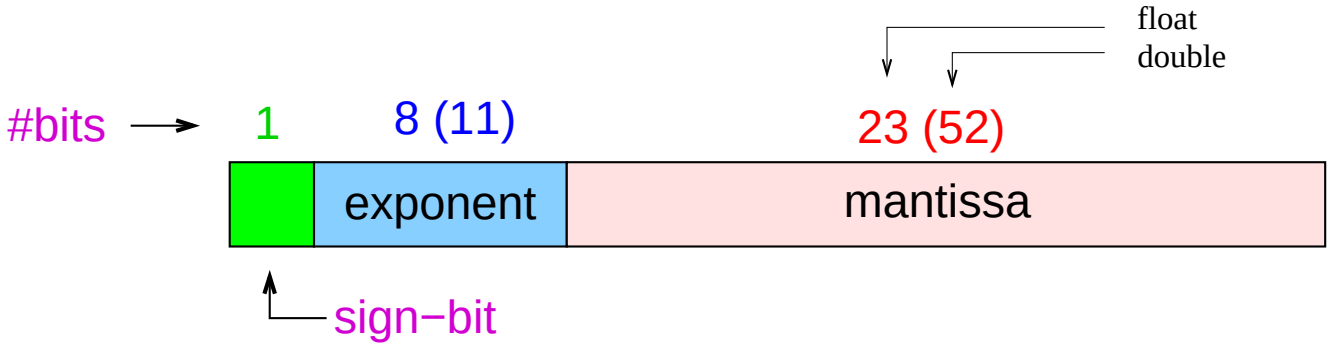
```
sizeof(bool) = 1
sizeof(char) = 1
sizeof(short) = 2
sizeof(int) = 4
sizeof(long) = 4
sizeof(long long) = 8
sizeof(float) = 4
sizeof(double) = 8
sizeof(long double) = 12
```

on a 64-bit machine

```
sizeof(bool) = 1
sizeof(char) = 1
sizeof(short) = 2
sizeof(int) = 4
sizeof(long) = 8
sizeof(long long) = 8
sizeof(float) = 4
sizeof(double) = 8
sizeof(long double) = 16
```

- Note that the figures may be different on your computer.
- A 32(64)-bit machine uses CPUs of which the data bus width and memory address width are 32 (64) bits.

Representation of Floating-Point Data



- Many programming language uses the IEEE 754 floating-point standard.
- Binary Representation of mantissa: e.g.

$$1.011_2 = 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

- Binary Representation of exponent: signed integer
- All floating-point data types in C++ are signed.
- ANSI C++ requires: size of float $\leq$ size of double

Question: Can every real number be represented by float in C++?

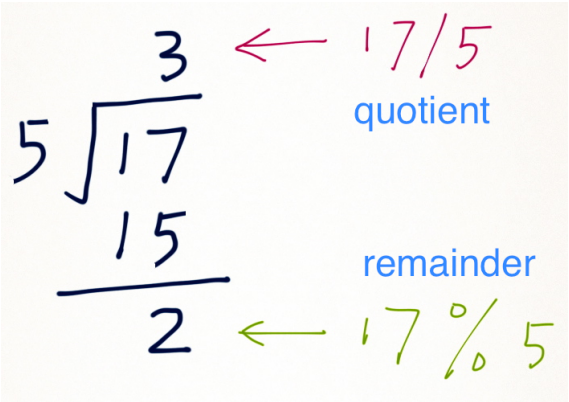
Underflow and Overflow in Integral Data Types

- Overflow: occurs when a data type is used to represent a number larger than what it can hold. e.g.
  - if you use a short int to store HK's population.
  - when a short int has its max value of 32767, and you want to add 1 to it.
- Underflow: occurs when a data type is used to represent a number smaller than what it can hold. e.g.
  - use an unsigned int to store a -ve number.

Underflow and Overflow in Floating-Point Data Types

- Underflow: when the -ve exponent becomes too large to fit in the exponent field of the floating-point number.
- Overflow: when the +ve exponent becomes too large to fit in the exponent field of the floating-point number.
- To prevent these from happening, use double if memory space allows.
- In fact, all floating literals (e.g., 1.23) is treated as double unless explicitly specified by a suffix (e.g., 1.23f).

Modulo Arithmetic



- mod is used to get the remainder in an integer division.

$$\text{mod}(17, 5) = 17 \bmod 5 = 17\%5 = 2$$

- Strictly speaking, $m \bmod n$ is defined only if n is +ve.
- Most programming languages support -ve divisor and different languages may give you different results!
- In C++, the modulo arithmetic is supported by the remainder operator % which allows -ve divisor.
- Question: What are the results of (-17)%5, 17%(-5), or (-17)%(-5)?

Priority Rules for the Usual Arithmetic Conversions for Binary Operations

- If either operand is of type long double, convert the other operand also to long double.
- If either operand is of type double, convert the other operand also to double.
- If either operand is of type float, convert the other operand also to float.
- Otherwise, the integral promotions shall be performed on both operands.
  - Similar rules are used for integral promotion of the operands.
  - Compute using integer arithmetic.

Question: What is the result of 3/4?